Lab Report 7: Flood Fill Algorithm

Name: _____ (Fill in your name)
Lab Program Number: 07
Date: _____ (Fill in your date)

Title

Understanding & Implementing Flood Fill Algorithm

Theory

The Flood Fill Algorithm is a classic technique in computer graphics used to fill an enclosed area of a specific color with a new color. It begins from a seed point and replaces all connected pixels of the **target color** (the "old" color) with the **fill color** until it meets a different color boundary. This algorithm is the foundation for tools like the "paint bucket" in image editing software.

Key Difference from Boundary Fill:

- **Boundary Fill**: Stops at a specific boundary color
- **Flood Fill**: Replaces all pixels of the same color as the starting pixel

Key Concepts:

- **Seed Point**: Starting point inside the region to be filled
- **Old Color**: The color to be replaced (target color)
- **Fill Color**: The new color to apply
- **4-Connected**: Considers only horizontal and vertical neighbors
- **8-Connected**: Considers all 8 surrounding pixels including diagonals

---

Advantages

- Can fill any enclosed area, regardless of shape complexity
- Handles irregular or complex internal patterns
- No need for a predefined boundary color

- Adaptable to different shapes and sizes
- Simple conceptual understanding

Disadvantages

- Can be slower for large fill areas due to recursion
- High memory usage with deep recursion stacks
- May fill unintended areas if not carefully managed
- More complex to implement efficiently than some alternatives
- Risk of infinite loops if fill color equals old color

Algorithm Steps

Flood Fill Algorithm (4-connected):

Step 1: Start
Initialize fill color, old color (color to replace), and seed point (x, y)

Step 2: Check the current pixel
Get the color of pixel at (x, y)

Step 3: Update pixel color
If pixel color equals the old color (and not fill color), change it to fill color

Step 4: Recursively apply to 4-connected neighbors
Process right, left, bottom, and top neighbors

Step 5: Repeat process
Continue until all connected pixels of old color are replaced

Step 6: Termination
Stop when no more connected pixels of the old color remain

Flood Fill Algorithm (8-connected):

Step 1: Start
Initialize fill color, old color, and seed point (x, y)

Step 2: Check the current pixel
Get the color of pixel at (x, y)

Step 3: Update pixel color
If pixel color equals the old color (and not fill color), change it to fill color

Step 4: Recursively apply to 8-connected neighbors
Process all eight directions including diagonals

Step 5: Repeat process
Continue until all connected pixels of old color are replaced

Step 6: Termination
Stop when no more connected pixels of the old color remain

Code

```c
```

```c
#include <stdio.h>
#include <graphics.h>
#include <math.h>

// Function to draw a hexagon
void drawHexagon(int centerX, int centerY, int radius) {
    int x[6], y[6];
    for (int i = 0; i < 6; i++) {
        x[i] = centerX + radius * cos(i * 60 * 3.14159 / 180);
        y[i] = centerY + radius * sin(i * 60 * 3.14159 / 180);
    }

    for (int i = 0; i < 6; i++) {
        line(x[i], y[i], x[(i + 1) % 6], y[(i + 1) % 6]);
    }
}

// 4-connected flood fill algorithm
void floodFill4(int x, int y, int fillColor, int oldColor) {
    // Check if current pixel is within screen bounds
    if (x < 0 || x >= getmaxx() || y < 0 || y >= getmaxy()) {
        return;
    }

    // Get current pixel color
    int currentColor = getpixel(x, y);

    // If pixel has the old color and is not already filled
    if (currentColor == oldColor && currentColor != fillColor) {
        putpixel(x, y, fillColor); // Fill the pixel

        // Recursively fill 4-connected neighbors
        floodFill4(x + 1, y, fillColor, oldColor); // Right
        floodFill4(x - 1, y, fillColor, oldColor); // Left
        floodFill4(x, y + 1, fillColor, oldColor); // Down
        floodFill4(x, y - 1, fillColor, oldColor); // Up
    }
}

// 8-connected flood fill algorithm
void floodFill8(int x, int y, int fillColor, int oldColor) {
    // Check if current pixel is within screen bounds
    if (x < 0 || x >= getmaxx() || y < 0 || y >= getmaxy()) {
```

```
        return;
    }

    // Get current pixel color
    int currentColor = getpixel(x, y);

    // If pixel has the old color and is not already filled
    if (currentColor == oldColor && currentColor != fillColor) {
        putpixel(x, y, fillColor); // Fill the pixel

        // Recursively fill all 8-connected neighbors
        floodFill8(x + 1, y, fillColor, oldColor); // Right
        floodFill8(x - 1, y, fillColor, oldColor); // Left
        floodFill8(x, y + 1, fillColor, oldColor); // Down
        floodFill8(x, y - 1, fillColor, oldColor); // Up
        floodFill8(x + 1, y + 1, fillColor, oldColor); // Bottom-right
        floodFill8(x - 1, y + 1, fillColor, oldColor); // Bottom-left
        floodFill8(x + 1, y - 1, fillColor, oldColor); // Top-right
        floodFill8(x - 1, y - 1, fillColor, oldColor); // Top-left
    }
}

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, ""); // Initialize graphics mode

    // Clear screen and set background
    cleardevice();

    // Demonstration 1: 4-connected flood fill on hexagon
    printf("Demonstrating 4-connected Flood Fill Algorithm...\n");
    printf("Drawing a hexagon...\n");

    // Draw a hexagon with white boundary
    int centerX = 250, centerY = 250, radius = 100;
    setcolor(WHITE);
    drawHexagon(centerX, centerY, radius);

    // Get a point inside the hexagon
    int seedX = centerX;
    int seedY = centerY;

    // Old color is BLACK (background), fill color is BLUE
    int oldColor = BLACK;
```

```
    int fillColor1 = BLUE;

    printf("Filling hexagon with blue using 4-connected method...\n");
    floodFill4(seedX, seedY, fillColor1, oldColor);

    getch(); // Wait for key press
    cleardevice(); // Clear screen for next demonstration

    // Demonstration 2: 8-connected flood fill on circle
    printf("\nDemonstrating 8-connected Flood Fill Algorithm...\n");
    printf("Drawing a circle...\n");

    // Draw a circle with white boundary
    setcolor(WHITE);
    circle(300, 250, 80);

    // Get a point inside the circle
    seedX = 300;
    seedY = 250;

    // Fill with different color using 8-connected method
    int fillColor2 = RED;

    printf("Filling circle with red using 8-connected method...\n");
    floodFill8(seedX, seedY, fillColor2, BLACK);

    getch(); // Wait for key press
    closegraph(); // Close graphics mode

    return 0;
}
```

Output

(Describe or attach your output here)

Example:
The program first displays a hexagon that gets filled with blue using the 4-connected flood fill. After pressing a key, it shows a circle filled with red using the 8-connected flood fill.

Visual demonstration:

- **First output**: White hexagon outline with blue interior (4-connected)
- **Second output**: White circle outline with red interior (8-connected)

The program demonstrates how flood fill replaces all connected pixels of the original background color with the new fill color, regardless of the boundary shape complexity.

Comparison: Flood Fill vs Boundary Fill

Aspect	Flood Fill	Boundary Fill
Stop Condition	Different from old color	Reaches boundary color
Use Case	Fill areas of same color	Fill within boundaries
Flexibility	More flexible	Requires defined boundary
Safety	Can fill unintended areas	Safer within boundaries

Conclusion

This lab successfully implemented both 4-connected and 8-connected Flood Fill Algorithms. The practical exercise demonstrated how flood fill differs from boundary fill by replacing all pixels of a specific color rather than filling until a boundary color is encountered.

Key learnings:

1. **Algorithm Behavior**:
    - The 4-connected version is suitable for filling areas without diagonal connections
    - The 8-connected version can fill through diagonal pixel connections
    - The "old color" serves as the fill criterion rather than a boundary color
2. **Critical Understanding**:
    - Flood fill uses the color at the seed point as the target to replace
    - It stops when it encounters any color different from the old color
    - This makes it ideal for filling uniform regions in paint applications
3. **Practical Applications**:
    - Paint bucket tools in image editors (Photoshop, GIMP)

- Game level editors for terrain filling
- Map coloring applications
- Image segmentation preprocessing

Limitations observed:

- Risk of stack overflow for large fill areas
- Potential to fill unintended areas if colors are similar
- Inefficient for very large regions compared to scan-line methods

Possible improvements:

- Implement iterative version using queue/stack to avoid stack overflow
- Add tolerance parameter for filling similar (but not exact) colors
- Implement scan-line flood fill for better performance on large areas

This exercise provided practical experience with fundamental region filling algorithms and their real-world applications in computer graphics, image processing, and interactive design tools.

Instructor's Signature: _____
Date: _____
Marks: _____ / _____